CURVINE
CURVATURE ENGINE

从存算分离到 Agent 数据底座

Curvine 的端云协同架构实践

多级分布式缓存 · 把对象存储读成本地 IO

CV
闾泽军 · Curvine Maintainer
OPPO 高级架构师 · Curvine Team

Apache 2.0 · CNCF Landscape
github.com/CurvineIO/curvine

CLOUD NATIVE COMPUTING FOUNDATION

DACon
DataFun.

目录

01 Data + AI 背景 对象存储成为默认数据层，数据流动效率成为关键 · ~5 min

02 Curvine 分布式缓存 缓存角色 · 场景全景 · 为什么是 Rust · 架构 · ~8 min

03 案例 · 落地实践 大数据热表 · 多模态检索 · 训推 KV Cache · Agent FS · ~18 min

04 展望 向 Agent 纵深 · 向高性能深处 · ~3 min

存算分离之后，瓶颈已经转移

存算分离成为云原生共识，对象存储成为默认数据层——AI 负载把「数据流动效率」推上了关键路径。

01

存算分离 · 云原生主流

计算弹性伸缩、存储集中沉淀；存算分离已是云上架构的默认答案。

02

对象存储 · 默认数据层

数据湖、训练语料、模型权重、Agent 工作区——都以 S3 兼容对象存储为家。

03

AI 负载 · 流动上关键路径

训练喂不饱 GPU、推理等 KV 复用、Agent 高频小 I/O——访问模式已从「大文件顺序」变为「高频、跨节点、小粒度」。

瓶颈不在「存了多少」，而在**「数据怎么流动」**——这正是「缝合式缓存方案」出现的背景，也是 Curvine 的起点。

Agent 数据底座：五类新问题

当 Agent 成为负载主体，数据底座要同时回答五个新问题。

01 万级工作区 创建风暴下，云盘 / EFS 供给跟不上

02 沙箱临时文件 高频随机读写，随用随弃

03 记忆与检索 向量 / 全文小 I/O，等不起延迟

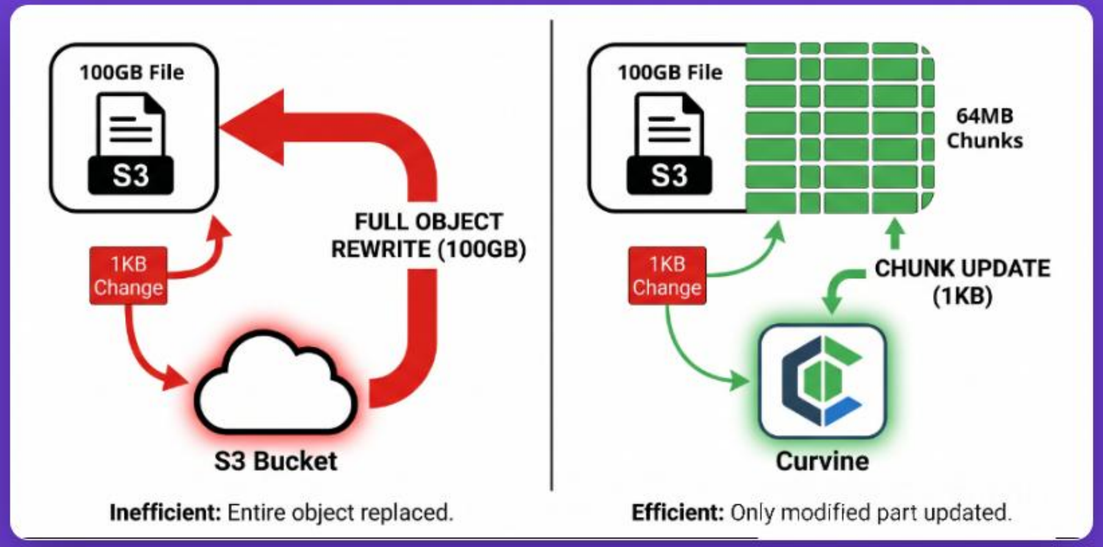
04 端云协同 两个执行面，要同一份工作区

05 交付产物 持久落对象存储，不随沙箱消失

五类问题，同一层底座——**这正是 Curvine 要解决的**。

缝合式缓存：两条先天代价

在对象存储前「缝」一层缓存——写放大与 NFS 网关瓶颈，调优解决不了。



对象更新机制对比：整对象重写 vs 分块更新

代价一

写放大

对象不可变——修改一个块，代价是**整对象重写**；大文件随机写被成倍放大，且按请求计费。

缝合式　改 1 块 → 整对象重写 → 全量同步

Curvine　改 1 块 → 分块写入 → 增量同步

代价二

NFS 网关瓶颈

NFS 单连接吞吐有限，数据与元数据都经**中心化网关**单点；并发一上来，这里就是天花板。

缝合式　客户端 → NFS → 中心网关 → 异步回源

Curvine　客户端 ⇄ 并行直连数据面

与其「缝合」，不如原生做一个文件系统

缝合式方案的三类缺口——语义、成本、结构；Curvine 的答案是一层原生缓存文件系统。

缝合式缓存方案

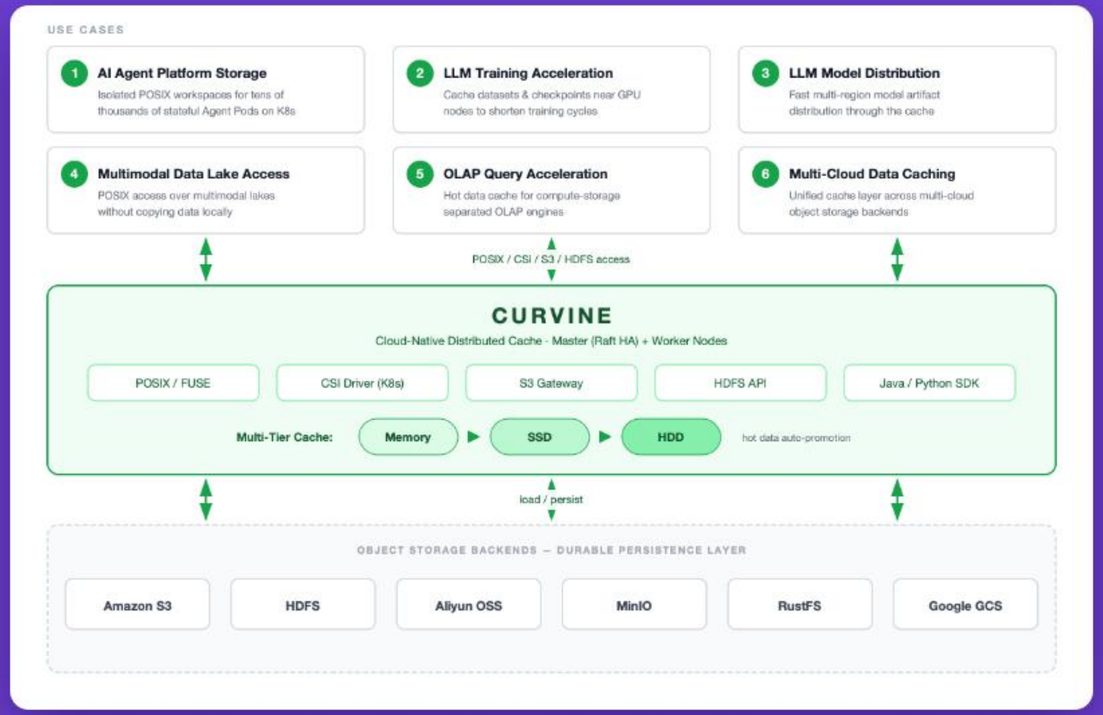
- **语义缺口** 原子重命名 / 硬链接 / xattr 不完整——git rename、node inotify、Python 打包语义不全
- **成本黑箱** S3 请求按次计费 + 写放大——隐性成本随流量放大
- **结构瓶颈** 中心化网关单点 + NFS 单连接——并发与吞吐受限

Curvine · 原生缓存文件系统

- **完整 POSIX** LTP 1129 项通过、0 Broken；git / node / inotify 原样运行
- **分块写入 + 增量同步** 读写路径并行直连，不经中心网关
- **缓存可丢 · 数据可恢复** 路径 1:1 映射对象存储，不做数据锁定

同一层**既做缓存、又做文件系统**：语义完整，路径最短。

场景全景：六个场景，一层底座



六类场景 → 同一层 Curvine（多级缓存）→ 对象存储 / HDFS / 多云后端

Curvine = 对象存储之上的缓存文件系统——数据持久在对象存储，访问在计算侧就近命中。

缓存角色

把对象存储读成本地 IO：Memory → SSD → HDD 三级分层，热数据自动提升

文件系统角色

完整 POSIX 命名空间 + 多协议接入：FUSE / CSI / S3 / HDFS / SDK

底座角色

六类场景共用一层：Agent 工作区 / 训练加速 / 模型分发 / 多模态 / OLAP / 多云缓存

为什么用 Rust 写存储：快 & 稳，不做取舍

存储系统既要延迟稳定、又要长期无人值守——Rust 让这两件事同时成立。

NO GC

延迟可预测

没有 Stop-the-World 暂停，尾延迟平稳——元数据 ~100µs 级、10 万+ QPS 稳定运行

MEMORY SAFETY

内存安全

所有权 + 借用检查在编译期消除悬垂指针、越界与 use-after-free——没有偶发 core dump

FEARLESS CONCURRENCY

无畏并发

Send / Sync 让数据竞争无法通过编译；Tokio 用少量线程承载海量 I/O

ZERO-COST

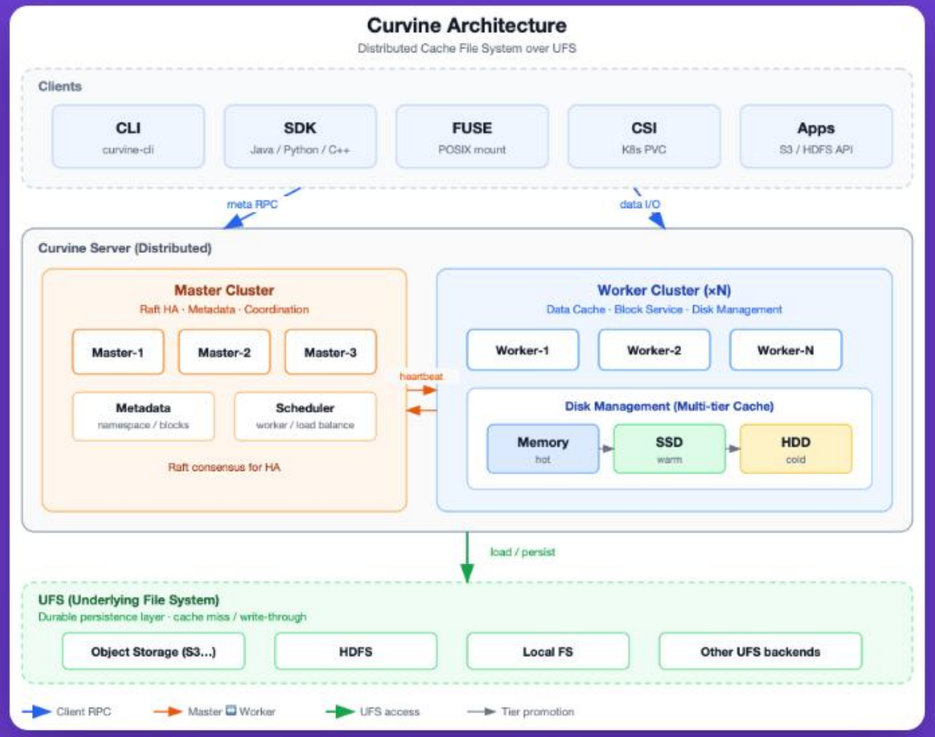
零成本抽象与零拷贝

泛型与 trait 编译期单态化；bytes 引用计数缓冲 + splice，数据路径少一次搬运

同一套 Rust 内核支撑 FUSE / CSI / S3 / SDK 全部接入面——**18 万+ 行代码，性能与稳定性来自同一个地基。**

架构：一条数据路径支撑多种负载

多级分布式缓存，把对象存储读成本地 IO——元数据走控制面，数据直连数据面。



元数据 → Master　数据 I/O → Worker 直连　同节点短路读 · 零网络

接入面

FUSE（POSIX）/ CSI（K8s PVC）/ S3 网关 / HDFS API / 多语言 SDK——同一数据路径

控制面 Master

Raft HA 元数据 + 调度，50 亿文件规模；元数据 ~100µs 级

数据面 Worker

块服务 + 多级缓存 Memory → SSD → HDD，热数据自动提升

统一命名空间

端、云、训练、推理看到同一棵目录树（Raft 强一致）

UFS 持久层

对象存储 / HDFS / 本地盘——缓存可丢，数据可恢复

同一层底座，四个落地案例

案例一 大数据热表

问题 OLAP 引擎跨云读热表——出口流量与首字节延迟双高

效果 对象存储下载流量、跨云带宽双双下降 **23%+**

案例二 多模态数据湖

问题 向量 / 全文检索是高频小 I/O，被首字节延迟支配

效果 检索 p50 快 **5.5×** · QPS **4.8×**（零侵入 2.3×）

案例三 训推 KV Cache

问题 长 prompt 重复 prefill、跨实例 KV 无法共享

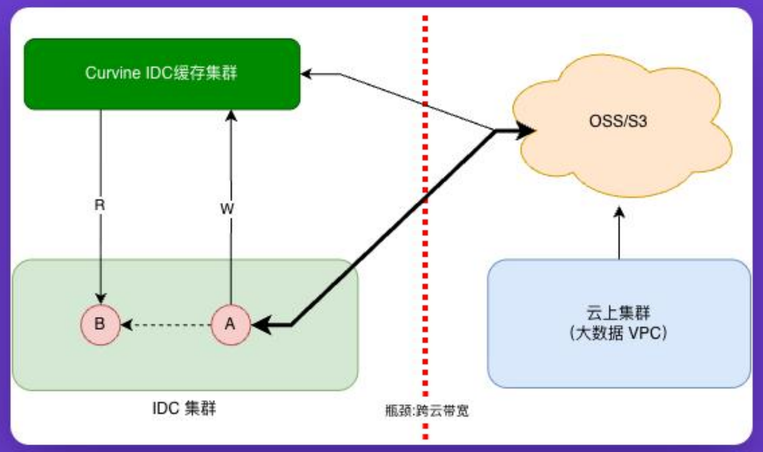
效果 TTFT 最多快 **2.7×** · MLPerf AU 八组胜六

案例四 Agent FS · 端云共享

问题 万级工作区供给、工具链要 POSIX、端云要同一棵树

效果 EKS **10,000 PVC 零 Pending**（AWS 官方博客实测）

案例一 大数据热表：跨云回源 → 本地命中



部署形态：云端集群写 OSS/S3 · IDC 侧经 Curvine 缓存读写（R / W 双路径）

问题

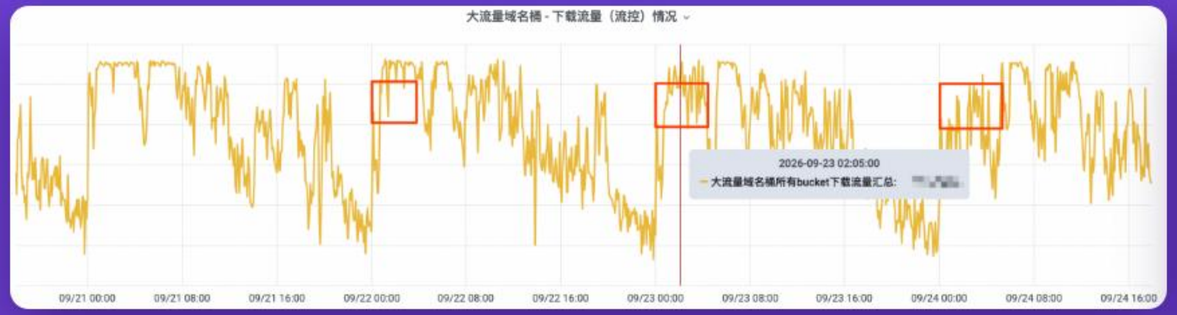
云端大数据集群把热表写到 OSS/S3；IDC 侧计算**每次读都跨云回源**——出口流量与首字节延迟双高，热表越热、回源越多。

方案

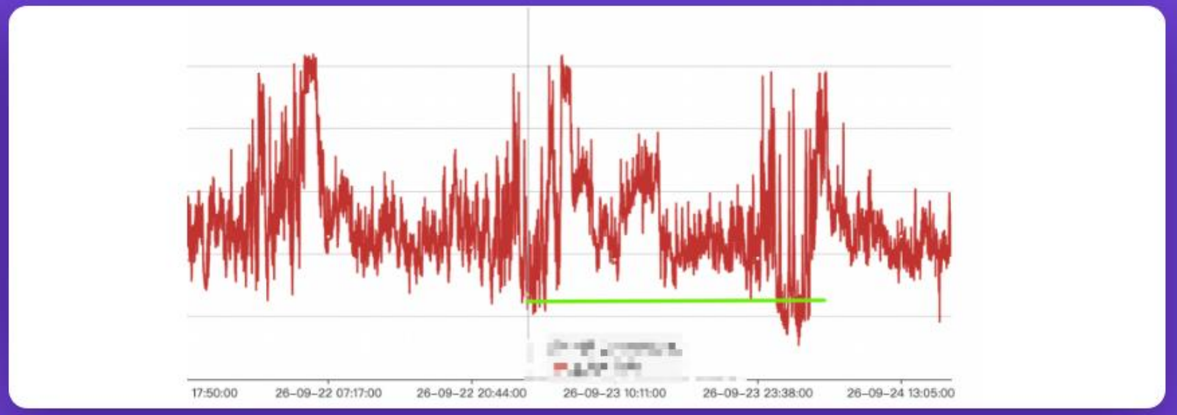
IDC 部署 Curvine 缓存集群，读写都经缓存：**R 读本地命中、W 经缓存汇聚回写**——跨云只剩按需回源与增量同步。

读 · IDC 就近命中　写 · 缓存汇聚增量同步　跨云 · 只留增量

案例一 热表接入后：对象存储流量与跨云带宽双双下降



对象存储 · bucket 下载流量（24h · 2026.09 · 内部实测）· 红框为接入后标段



跨云带宽（24h · 2026.09 · 内部实测）· 绿线为接入后稳态区间

↓ 23%+	↓ 23%+	0
跨云带宽（灰度对比）	对象存储下载流量（灰度对比）	业务侧改动——读写仍是 POSIX 原路径

案例二 多模态数据湖：慢在首字节，不在算力

envelope · 100 万条 1536 维向量（dbpedia-entities-openai-1M）· 每组 10,000 次查询 · LanceDB 0.34.0

问题

向量 / 全文检索 = 几十到几百次**小粒度随机读**；对象存储上每次 I/O 都背一个首字节延迟——p50 11ms、p99 62ms，QPS 卡在 69。

方案 · 两档接入

1 **零侵入 · 只放 _indices 目录**

不改一行业务代码——LanceDB 的索引目录指向 Curvine，全文检索立刻 2.3×

2 **全程 curvine:// · 收益最大化**

curvine:// 作为 LanceDB ObjectStore 直读——数据不迁移，POSIX / SDK 双通道



多模态记忆：图片 / 视频 / Embedding 以 **Lance / Parquet** 落盘，检索走缓存直读——**不复制数据、不换引擎**。

案例二 检索效果：p50 5.5× · QPS 4.8× · p99 收敛 6×

11→2 ms
向量检索 p50 · 5.5×

69→333
检索 QPS · 4.8×

62→10 ms
p99 尾延迟收敛

2.3×
零侵入（只放 _indices 目录）

查询类型	OSS	只缓存索引	全程 Curvine
vector 向量检索	69.4 · 11ms · 62ms	76.9 · 9ms · 51ms	**333.3 · 2ms · 10ms**
fts 全文检索	31.4 · 26ms · 88ms	71.4 · 12ms · 37ms	**144.9 · 5ms · 15ms**
hybrid 混合检索	25.2 · 36ms · 90ms	37.9 · 24ms · 52ms	**73.5 · 13ms · 21ms**
vector_with_filter	2.6 · 377ms	2.6 · 378ms	3.9 · 249ms（仅 1.5×）

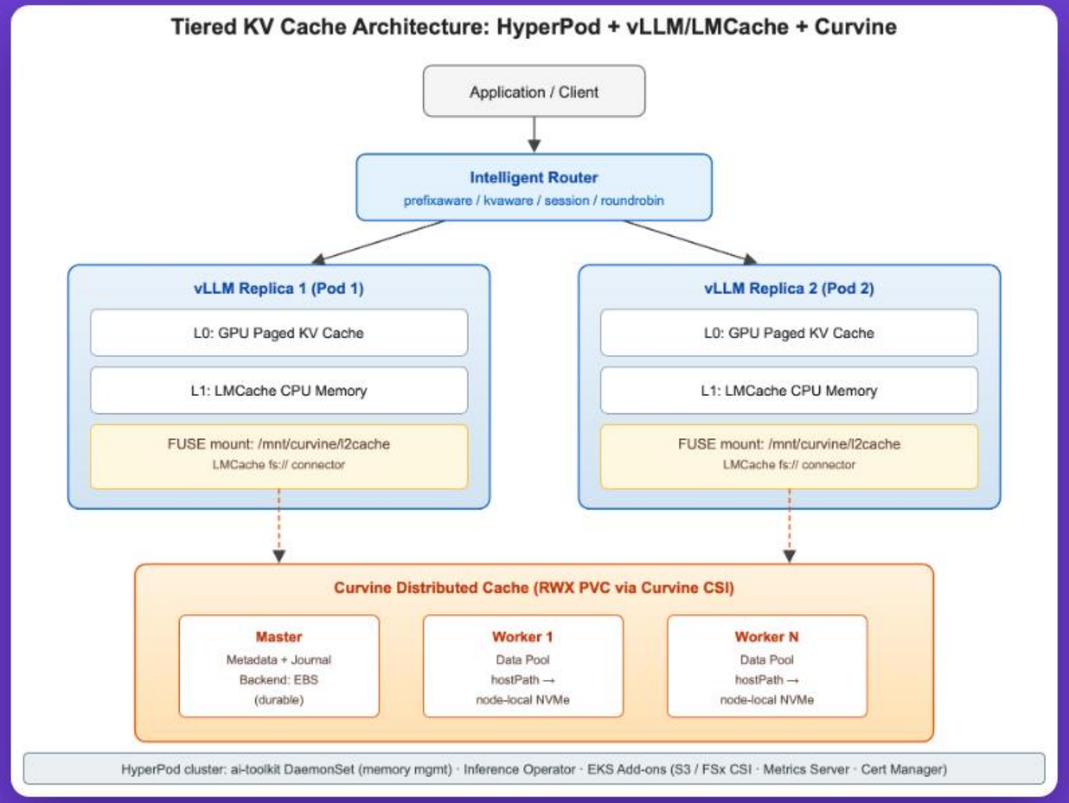
单元格 = QPS · p50 · p99

如实标注 · 唯一不显著的一组

vector_with_filter 仅 1.5×——瓶颈在过滤 + 扫描的 CPU 路径，不在 I/O。知道技术在哪不起作用，和知道它在哪起作用同样重要。

环境为最小集群 1 Master + 1 Worker——这是下限，不是上限

案例三 训推 KV Cache：跨实例复用，跳过重复 prefill



SageMaker HyperPod · vLLM + LMCache fs:// · Curvine RWX PVC (Master + 多 Worker)

问题

长上下文推理中 prefill 占大头；而 KV 只活在**单实例显存**里——每个副本、每次会话都从头重算。上下文越长，TTFT 越贵。

方案 · 三级分层

L0 GPU HBM · Paged KV

显存内分页缓存，推理引擎原生能力

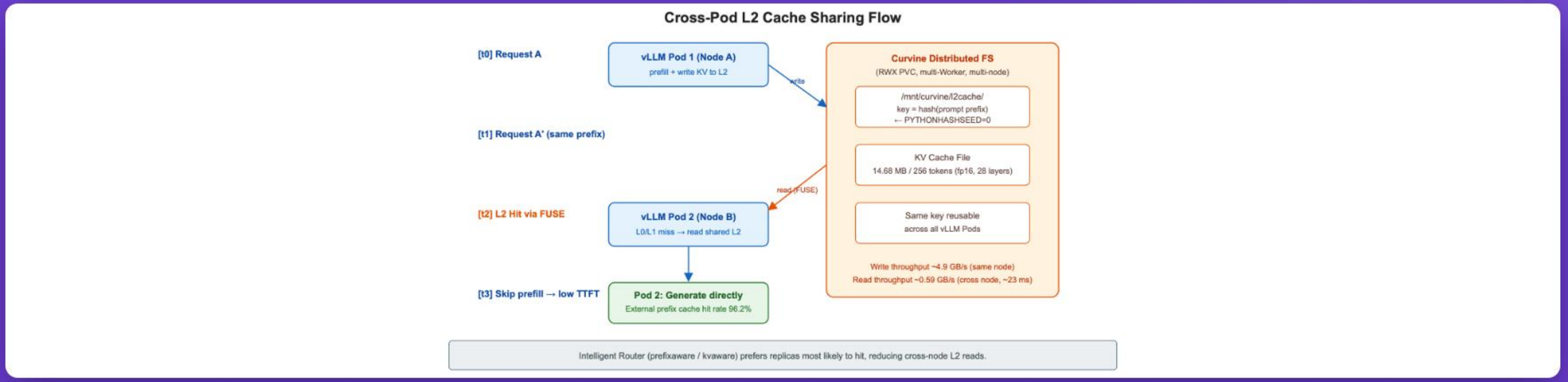
L1 CPU DRAM · LMCache

节点内存池，进程内外都可读

L2 共享 NVMe · Curvine

汇聚各节点本地盘为 RWX 共享层——fs:// 直挂，跨 Pod 可见

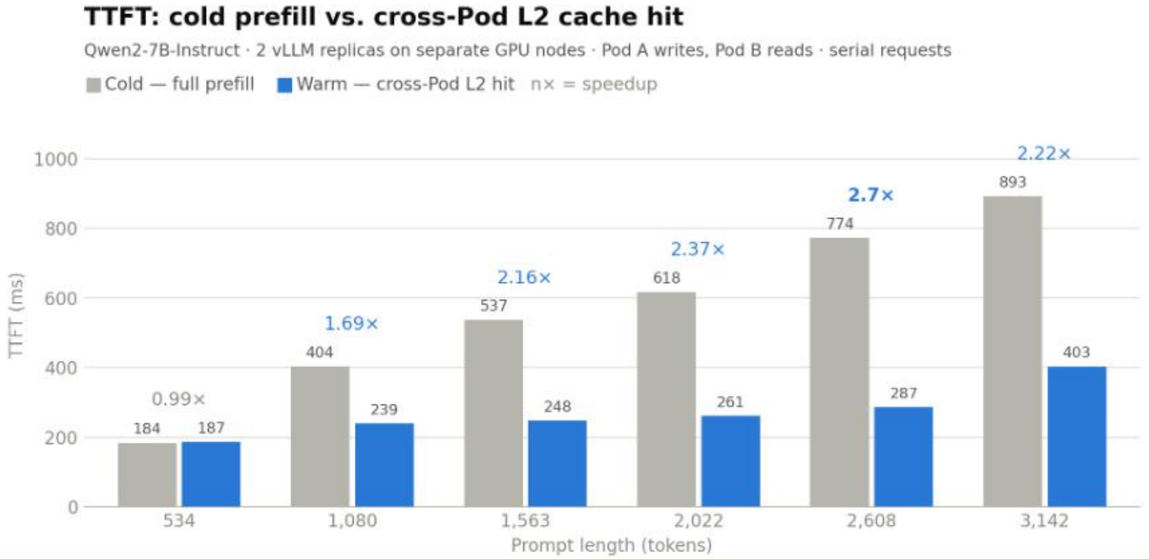
案例三 跨 Pod 复用：一次 prefill，全副本可命中



t0 Pod1 prefill 写 L2 → t1 同前缀请求 → t2 Pod2 L0/L1 未命中、FUSE 读共享 L2 → t3 直接生成（跳过 prefill）· key = hash(prompt prefix)，跨全部 vLLM Pod 复用

96.2%	~4.9 GB/s	~0.59 GB/s	14.68 MB
外部前缀缓存命中率	L2 写吞吐（同节点）	L2 读吞吐（跨节点 · ~23ms）	KV 文件 / 256 token（fp16 · 28 层）

案例三 TTFT 效果：上下文越长，收益越大



Qwen2-7B-Instruct · 2× vLLM 跨 GPU 节点 · Pod A 写 / Pod B 读 · 串行请求 · 灰 = 冷 prefill，蓝 = 跨 Pod L2 命中

2.7×
峰值加速 · 2,608 token（774→287ms）

2.16 → 2.7×
1.5K–2.6K token 区间加速带

0.99×
534 token 短上下文——基本无收益

DACon
DataFun.

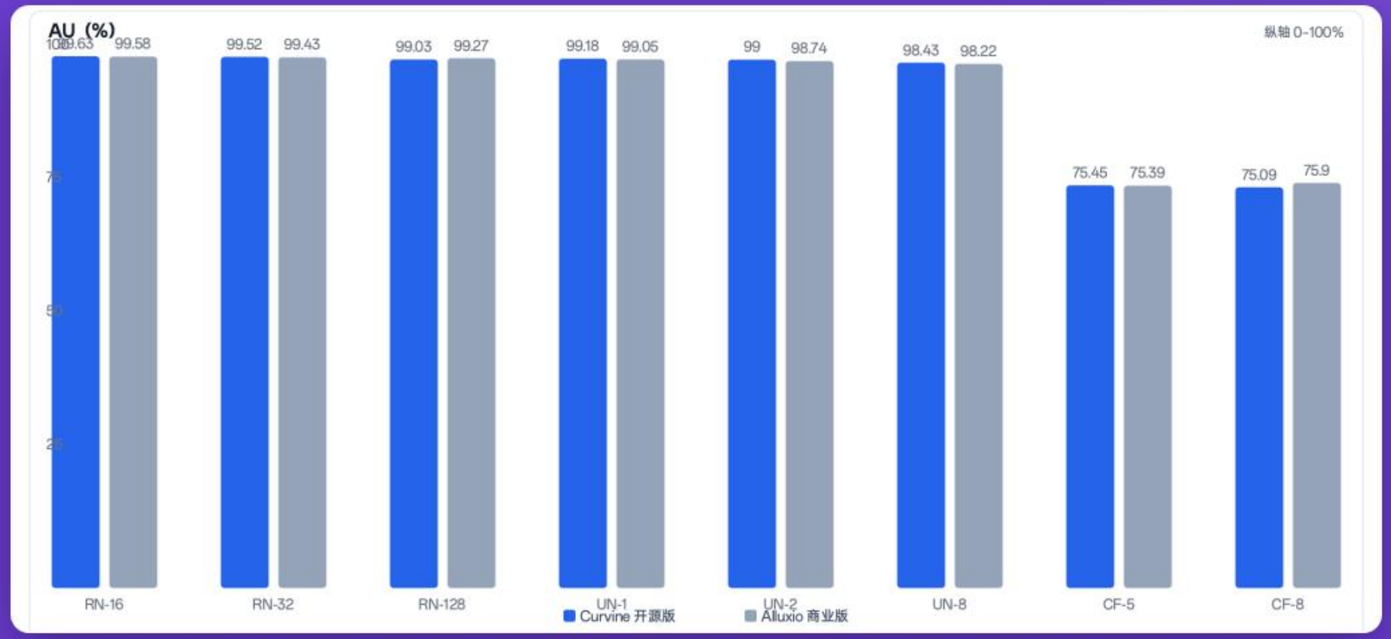
案例三 MLPerf Storage 在测什么：存储能不能把加速器喂饱

不测算力——DLIO 按真实 batch 读数据，计算按 H100 实测时间仿真，数据真走 POSIX / FUSE 才算一次有效供给。

DLIO · 真实 batch 读数据 | H100 实测时间 · 仿真计算 | AU = 模拟计算 ÷ 总墙钟 | 门槛 ≥ 90 / ≥ 90 / ≥ 70



案例三 MLPerf 对标：把加速器喂得更饱



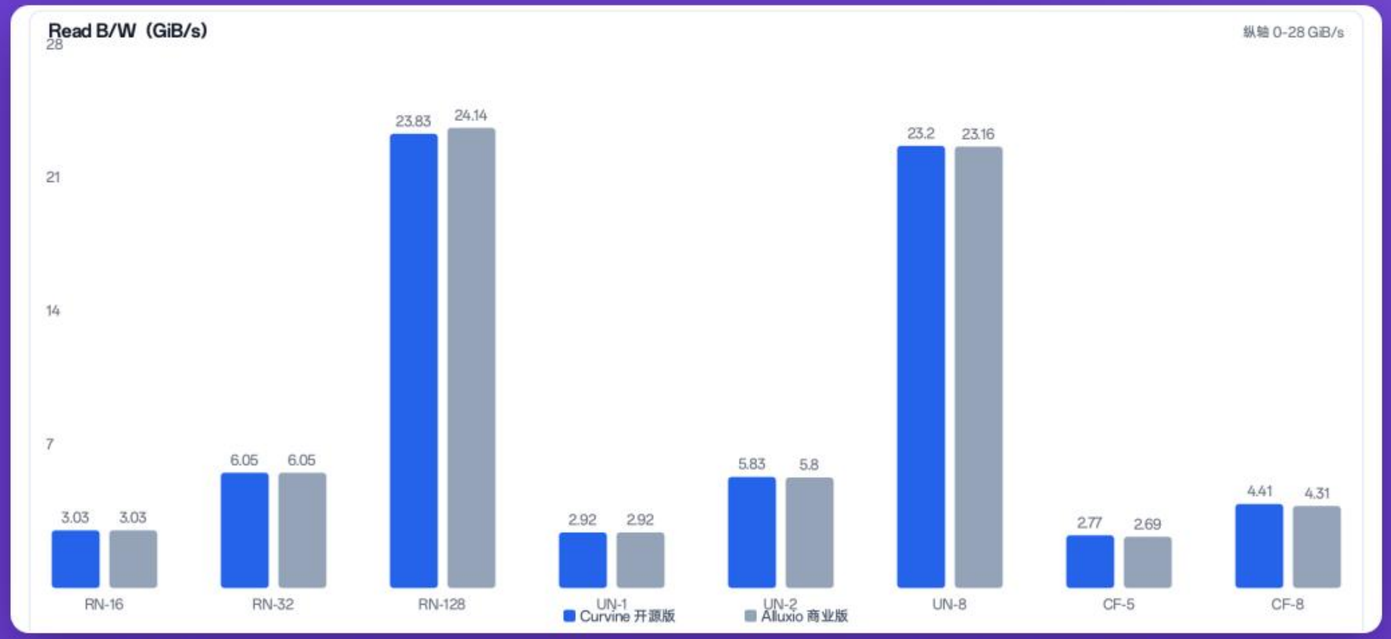
加速器利用率 AU · ResNet-50 / U-Net3D / CosmoFlow 八组 · AU 门槛 ≥90 / ≥90 / ≥70

6 / 8	3 / 3	6 / 8 · 6
AU 胜出（八组对标）	U-Net3D 三档规模全部超过	八组全部跨过 AU 门槛（≥90 / ≥90 / ≥70）

Curvine 为内部对标（非官方提交）· Alluxio 为 MLCommons Closed 官方提交（Enterprise AI）

DACon
DataFun.

案例三 聚合带宽：七组不落后，唯一的 −1.3% 如实标注



聚合读带宽 · 单位 GiB/s · 八组对标 · CosmoFlow 8 卡两侧 Client 配比不同（原文明示，不单独解释）

7 / 8
聚合读带宽不落后

-1.3%
唯一落后：ResNet-50 · 128 卡（23.83 vs 24.14 GiB/s）

3 / 3
U-Net3D 三档带宽均高于对照

内部对标（非官方提交）· 带宽统一 GiB/s

DACon
DataFun.

案例四 Agent 文件系统：万级工作区的供给与共享

为什么传统方案不行

方案	瓶颈
EBS 云盘	每节点约 28 块盘上限——100 Pod/节点密度需 4 倍节点
EFS 动态供给	创建风暴下 API 限流——PVC 排队、供给变慢
S3 挂载	非 POSIX——git / node / vite 跑不起来
端云各一套存储	视图不一致——无法共享同一份工作区

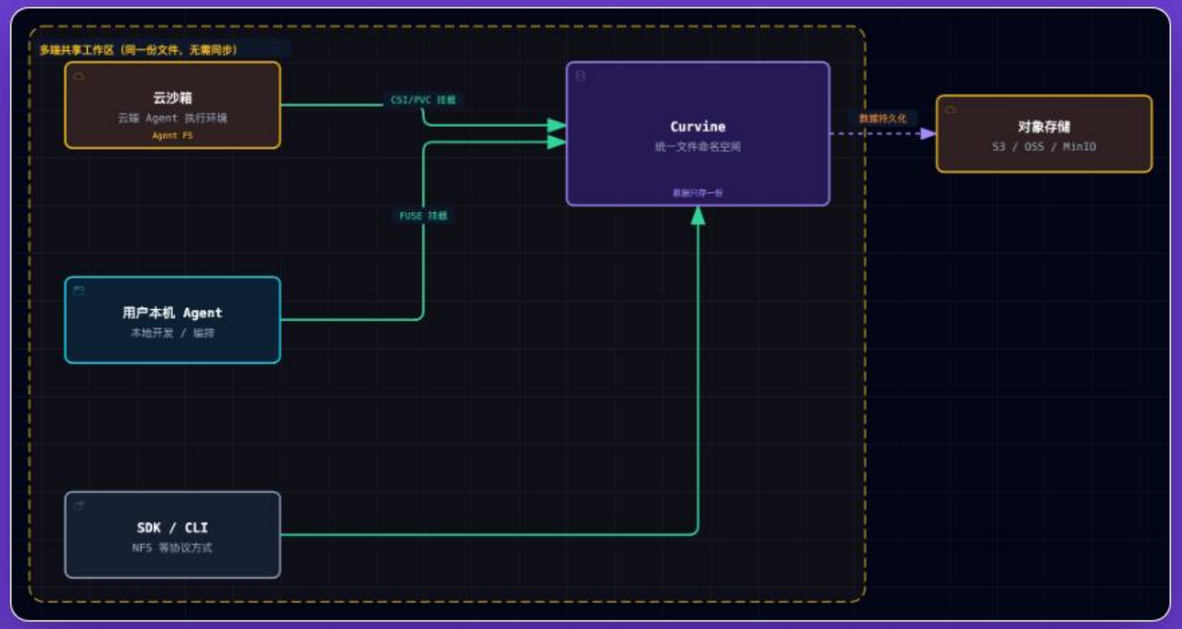
一线观察：真实 Agent 工作区 = SOUL.md / AGENTS.md / MEMORY.md + node_modules + .git——天然的文件系统负载

Curvine 的做法

- **CSI 动态供给** 工作区 = 共享命名空间上的一次 mkdir——毫秒级、不调用云 API、无单节点上限
- **FUSE 完整 POSIX** git / node / vite / inotify 原样运行（LTP 1129 项）
- **统一命名空间** 端、云、沙箱挂载同一棵目录树（Raft 强一致）
- **两类存储一份供给** scratch 临时（随沙箱生灭）+ 工作区持久（跨重建保留）

Agent 的工作区不是一个云盘，而是**一棵目录树**——执行是瞬态的，工作区是持久的。

案例四 端云协同：两个执行面，一棵目录树



三端接入 · 同一份文件 · 数据持久化到对象存储

临时文件 · 高频随机读写

agent 执行过程中的 scratch 小 IO——高频随机读写并行直连数据面，就近命中

目标产物 · 持久化

中间产物与交付物直接落盘交换（A2A artifact）；**目标产物持久化到对象存储**——沙箱销毁不丢

统一命名空间

端、云、沙箱挂载同一棵目录树（Raft 强一致）——同一份文件，无需同步

共享只读层

代码仓库 / skills / node_modules 一次准备、多端复用

执行面可以在端、在云、在沙箱里随时生灭——**工作区始终是同一棵树**。

案例四 EKS 实测：10,000 个工作区，零 Pending

10,000/10,000	~100	4	~300TB
PVC 全部 Bound · 零 Pending / 零 Failed	Agent Pod / 节点 · 零存储争抢	核心 Pod = 1 Master + 3 Worker	单集群供给容量（30Gi 最小分配 × 万级）

持久性实测

pod-0 / pod-5000 / pod-9999 三个样本写入——**Pod 重启后数据仍在，跨节点视图一致**。工作区不是缓存的生命周期，而是数据的生命周期。

为什么只有 4 个核心 Pod

元数据走 Master、数据直连 Worker——**控制面极轻**；存储容量随 Worker 本地盘横向扩展，不靠堆云盘。

一张访问面，两种模式：FSMode 与 CacheMode

同一套内核、同一个命名空间——按工作负载选择模式，数据始终不被锁定。

模式以挂载点为粒度控制——同一集群内灵活配置、按需混用。

`FSMode · 完整 POSIX 文件系统`

语义完整

LTP 1129 项通过 · 0 Broken（141 TCONF 跳过）

强一致元数据

Raft HA · 50 亿文件 · rename 4.3 万 QPS

面向

Agent 工作区 / 工具链 / 需要完整文件语义的负载

`CacheMode · 弹性缓存层`

就近命中

Memory → SSD → HDD 多级缓存，回源对象存储

弹性可丢

缓存失效不影响数据——路径 1:1 映射对象存储，随时可恢复

面向

热表加速 / KV Cache / 模型分发 / 跨云缓存

不是两套系统：**同一个内核、同一个命名空间**——该要语义的地方给语义，该要速度的地方给速度。

下一步：向 Agent 纵深，向高性能深处

AGENT 数据底座 · 纵深

- **沙箱与工作区** 万级密度下的快速供给、快照与恢复
- **记忆与检索** 向量、全文与文件收敛到同一命名空间
- **推理延伸** KV Cache 与模型分发共用同一个缓存层

存储底座 · 高性能

RDMA 高速网络 | io_uring 异步 IO | SPDK 用户态存储 | 零拷贝与元数据路径

目标只有一个：让「**把对象存储读成本地 IO**」更快、更稳。

Built with Rust · Built for Speed

Curvature Engine For Data&AI

Thank You

从存算分离到 Agent 数据底座——与 Curvine 一起，把对象存储读成本地 IO

GITHUB · APACHE 2.0

github.com/CurvineIO/curvine

Star · Issue · PR 都欢迎

DOCS & BLOG

curvineio.github.io

文档 · 博客 · 各案例接入指南

WECHAT · 微信公众号



扫码关注

CLOUD NATIVE COMPUTING FOUNDATION

现场问答时间